

PART IV — FROM RAG TO AGENTIC RAG

Chapter 20B

Structured Output Reliability with `fix_llm_output.py`

“A model that fails to produce valid JSON has not failed to reason — it has failed to format, and formatting can be fixed downstream of thought.”

Chapter 20B — Structured Output Reliability with `fix_llm_output.py`

This chapter is a full accounting of that module: the forty distinct ways a real LLM has been observed to fail at emitting JSON, the five-stage pipeline that recovers from as many of them as possible without ever inventing data, the Pydantic schemas that give each of ten different call sites its own validated shape, and the test harness that turns "seems to work" into three hundred and two checked cases.

20B.1 The forty failure modes

Definition — Structured-output failure mode

A specific, reproducible way a language model's response fails to satisfy "valid JSON matching the expected schema," distinct enough from other failure modes to require its own detection and, where possible, its own recovery path.

Forty distinct failure modes were catalogued through literature review, community bug reports, and direct observation of this project's own LLM calls — and they group cleanly into six families, which is what makes a layered repair pipeline tractable instead of an ever-growing pile of special cases.

Family	Examples
Syntax failures	Broken JSON, truncation, trailing commas, single quotes, unquoted keys
Structural failures	Wrong top-level type, list-wrapped dict, partial schema hallucination
Wrapper failures	Markdown fences, code blocks, <code>submit_answer({...})</code> -style calls, blockquotes
Language failures	Python <code>True`/None`</code> , JS comments, YAML, XML, class/attribute syntax
Semantic failures	Hallucinated placeholders, refusals, multiple candidate answers, infinite repetition
Encoding failures	Unicode corruption, stray control characters

No single technique closes all six families — a regex fixes a wrapper, not a hallucinated placeholder; a tolerant parser recovers truncated syntax, not a refusal message wearing JSON's clothing. That mismatch, not any one clever trick, is the actual argument for Section 20B.2's layered design.

20B.2 The layered repair pipeline

Every call to `fix_llm_output` walks the same five stages shown in Figure 20B.1, in the same order, each one cheaper and more trustworthy than the one after it, escalating only when the current stage actually fails rather than running every stage regardless.

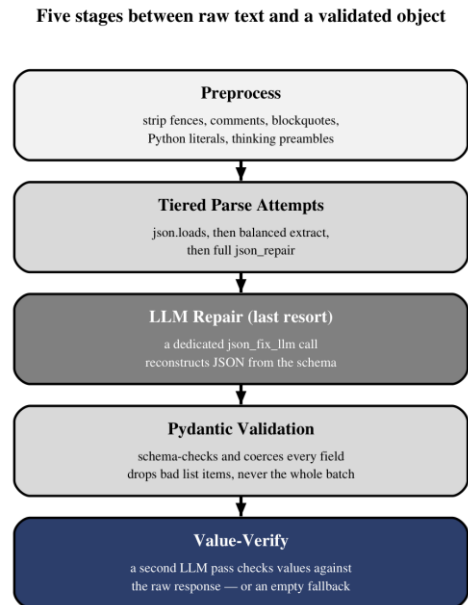


Figure 20B.1 — Cheap, deterministic recovery is attempted first; an LLM is only asked when it has to be.

```
def fix_llm_output(expected_output, raw_response, correct=False, llm=None,
config=None):
    model, top_level = _resolve_expected(expected_output)
    obj = _parse_to_python(raw_response)          # preprocess -> tiered
    parse
    for _ in range(_JSON_REPAIR_TRIES):
        if obj is not None:
            break
        obj = _LLM_Json_Repair(raw_response, model, top_level,
config=config)
        if obj is None:
            return _empty(top_level), False
        obj = _coerce_top_level(obj, top_level)
        validated = _validate_with_pydantic(obj, model, top_level)
        if validated is None:
            return _empty(top_level), False
    return _Verify_And_Correct(validated, raw_response, config=config),
    True
```

The ordering is the design: a tolerant parser is tried before an LLM call because it is thousands of times cheaper when it works, and it works often enough — Section 20B.11's own measurements show most real responses never reach the LLM-repair tier at all.

20B.3 Preprocessing

Six cheap, lossless text transforms run before any parse attempt, each one closing off exactly one wrapper failure from Section 20B.1's catalogue: strip code fences, strip markdown blockquotes, unwrap a function-call-style wrapper, strip JSON comments, convert Python literals to JSON ones, and cut a "thinking" preamble that precedes the actual JSON.

```
def _fix_python_literals(s: str) -> str:
    """Replace True/False/None with true/false/null — but only outside
    strings,
    walked character by character so 'a True story' is never corrupted."""
    out, i, in_str, str_char = [], 0, False, ""
    while i < len(s):
        ch = s[i]
        if in_str:
            out.append(ch)
            if ch == str_char:
                in_str = False
            i += 1
            continue
        if ch in ('"', "'"):
            in_str, str_char = True, ch
            out.append(ch); i += 1; continue
        for py, js in (("True", "true"), ("False", "false"), ("None",
            "null")):
            if s.startswith(py, i):
                out.append(js); i += len(py); break
        else:
            out.append(ch); i += 1
    return "".join(out)
```

Common pitfall — A naive regex replacing Python literals anywhere it finds them

A regex substitution for ``True` → `true`` with no awareness of string boundaries will happily corrupt a legitimate string value like ``"a True story"`` into ``"a true story"`` — a silent, low-visibility data change rather than a parse failure, which is worse: parse failures get noticed. `_fix_python_literals`` walks the text one character at a time, tracking whether it is currently inside a quoted string, and only replaces a literal when it is not — the same string-aware scanning discipline Section 20B.4 uses for a harder version of the identical problem.

20B.4 Balanced top-level JSON extraction

Definition — Balanced extraction

Locating the first top-level `{` or `[` in a text and finding its true matching closing bracket by depth-counting, rather than by pattern-matching to the last `}` or `]` in the string — the only way to correctly isolate a JSON object from surrounding prose that may itself contain brackets.

A greedy regex like ``\{.*\}`` fails the moment a model appends even one sentence of trailing prose after the JSON, because ``.*`` happily swallows past the true closing brace looking for a later one. ``_extract_balanced_json`` instead counts nesting depth character by character, and — critically — suspends counting entirely while walking through a quoted string, so a stray `{` or `}` typed inside a string value never miscounts the real structure's depth.

```
def _extract_balanced_json(text: str) -> str | None:
    start, open_ch = next(((i, c) for i, c in enumerate(text) if c in
        "{[", (-1, ""))
    if start == -1:
        return None
    close_ch = "}" if open_ch == "{" else "]"
    depth, in_str, str_char, i = 0, False, "", start
    while i < len(text):
        ch = text[i]
        if in_str:
            if ch == str_char:
                in_str = False
            elif ch in ("'", '"'):
                in_str, str_char = True, ch
        elif ch == open_ch:
            depth += 1
        elif ch == close_ch:
            depth -= 1
            if depth == 0:
                return text[start:i + 1]
        i += 1
    return None # never balanced — let the repair layer handle truncation
```

Returning ``None`` on an unbalanced structure is itself a deliberate choice, not an oversight — a truncated response has no true closing bracket to find, and inventing one here would mean silently guessing at how the response was supposed to end. That guess belongs to ``json_repair`` (Section 20B.5), a tool built specifically for tolerant reconstruction, not to a function whose entire job is precise, honest boundary detection.

20B.5 json_repair as the tolerant fallback

Where balanced extraction is precise and refuses to guess, `json_repair` is the opposite by design — a permissive parser that reconstructs plausible JSON from truncated, malformed, or loosely-structured text, invoked only after the precise tiers have already failed.

```
def _try_repair(s: str) -> Any | None:
    try:
        result = repair_json(s, return_objects=True)
        return None if result in ("", None) else result
    except Exception:
        return None
```

Trust it for what it is good at — truncation, unbalanced brackets, stray prose between JSON candidates — and distrust it for exactly the failure mode Chapter 20.11 already named: a permissive tool asked to reconstruct structure from text that never contained real data will reconstruct **something**, and something plausible-looking is more dangerous than an honest parse failure, because it does not look like a failure to whatever consumes it next. `json_repair` is a recovery tool for damaged JSON, not a substitute for data that was never there — Section 20B.8's empty-fallback principle is what keeps that distinction from collapsing.

20B.6 Project-specific Pydantic schemas

Ten schemas cover every structured call site in the pipeline, registered by a short string tag so a caller never has to import a Pydantic class directly — just name what it expects and hand over the raw text.

Tag	Shape	Used by
<code>`merge`</code>	dict	Chunk-merge output (Chapter 22B's NAC stage)
<code>`dc_scan`</code>	list	Redundancy-group proposals
<code>`redundancy_judge`</code>	list	Per-group CONFIRMED / REJECTED verdicts
<code>`retrieval_judge`</code>	dict	Per-chunk relevance verdicts (Chapter 12)
<code>`merge_judge`</code>	dict	Faithfulness verdict on a merge
<code>`lbc_compress`</code>	dict	Query-focused chunk compression output
<code>`lbc_judge`</code>	dict	Compression-safety verdict
<code>`grounding_judge`</code>	dict	The multi-verdict answer judge (Chapter 20.7)
<code>`distill_qa`</code>	list	Self-learning distillation Q&A pairs
<code>`query_variants`</code>	list	Reformulated query strings

Every schema does more than reject bad shapes — its field validators actively coerce recoverable ones. `MergeSchema.merged_from` accepts an integer, a numeric string, or `None`, and normalizes all three to a real `int` rather than rejecting anything that isn't already exactly the right type:

```
class MergeSchema(_BaseStrict):
    content: str
    sources: list[str] = Field(default_factory=list)
    merged_from: int = 0

    @field_validator("merged_from", mode="before")
    @classmethod
    def _coerce_merged_from(cls, v):
        if v is None or v == "":
            return 0
        try:
            return int(v)
        except (TypeError, ValueError):
            return 0
```

A schema this permissive at the field level and this strict at the shape level is the balance the whole module is built around: never reject a value that could obviously be coerced, never accept a shape that plainly does not match.

20B.7 The correct parameter — a signature that outgrew its own behavior

`fix_llm_output`'s signature carries a `correct: bool = False` parameter, and its name and default suggest exactly the strict-versus-aggressive-recovery split you would expect: `False` for validation only, `True` to additionally opt into the value-verification pass. Reading the function body tells a different story — `_Verify_And_Correct` runs unconditionally, every time, regardless of what `correct` is set to. The parameter is accepted and never once referenced again.

Common pitfall — A parameter that looks load-bearing and isn't

This is Chapter 19B.13's dead-parameter pattern again, in a different file: a signature that documents an intention — `correct` implies a choice exists — that the implementation quietly stopped honoring at some point in the module's evolution, almost certainly when value-verification changed from optional to standard behavior without anyone removing the now-meaningless flag. Every call site in the codebase still passes some value for `correct`, and every one of those values is silently ignored. Before assuming a boolean flag changes behavior, grep for where it's actually read, not just where it's declared — a parameter's name is a claim, not a guarantee.

The honest fix mirrors ADR-055's from Chapter 19B exactly: either remove the parameter and let every call site reflect what actually happens, or make it real again by gating `_Verify_And_Correct` behind it. Leaving it as-is costs nothing at runtime and costs a reader real confusion the first time they trust the signature over the implementation.

20B.8 The empty-fallback principle

Definition — Empty-fallback principle

On any unrecoverable failure — parsing, coercion, or schema validation — return the schema's empty container (`{}` for a dict-shaped schema, `[]` for a list-shaped one) rather than raising an exception or returning partially-invented data.

Every hard-failure exit in `fix\_llm\_output` returns the same shape of thing: `\_empty(top\_level), False` — an empty container paired with an explicit boolean signaling failure, never a crash and never a guess.

```
def _empty(top_level: str) -> Union[dict, list]:  
    return {} if top_level == "dict" else []
```

The choice not to raise is deliberate: a validator that raises turns every malformed judge response into a pipeline-halting exception, when the correct response to "the judge failed to produce a verdict" is usually "treat this as no verdict and move on" — exactly the `UNKNOWN`-defaults-closed behavior Chapter 20.7 built into the grounding judge. An empty list is also, separately, a legitimate real answer — "no redundancy groups found" is a correct, non-error result that happens to look identical to a parse failure's fallback shape, which is precisely why the function returns a success boolean alongside the value instead of overloading emptiness itself as the failure signal.

20B.9 The Outlines framework

Definition — Constrained decoding

Restricting a model's token generation at each step to only the tokens that keep its output consistent with a target grammar or schema, implemented via masking the model's own logits — a guarantee available only when the caller controls the actual inference loop.

Outlines was evaluated specifically for guaranteeing schema-valid JSON and rejected for the same structural reason both times it came up: its finite-state-machine token masking requires direct access to the model's logits during generation, which only exists for a locally-hosted model. Point it at a remote OpenAI-compatible API — Groq included — and `outlines.from\_openai()` cannot mask anything; it silently falls back to sending the same `response\_format: json\_schema` parameter a caller could set directly, with an added dependency and, for complex schemas, a schema-compilation cost measured in minutes rather than milliseconds.

The rejection was not a verdict on Outlines' technique — constrained decoding is real and effective where it can actually run. It was a verdict on this project's deployment shape: a remote, API-based inference provider that no dependency can retrofit local guarantees onto. Local model hosting with Outlines and vLLM was named explicitly as the path that would make the library's real capability available, and explicitly deferred rather than pursued.

20B.10 Why response_format: json_schema only works on certain models

Native `json\_schema` mode is the actual server-side enforcement Section 20B.9 concluded was the correct approach for remote inference — but "available" and "available on the model you are currently using" are different claims, verified separately. On Groq specifically, `llama-3.3-70b-versatile` supports it; `llama-3.1-8b-instant`, the development-tier model Chapter 13.3 chose for iteration speed, does not.

```
response = llm.invoke(
    messages,
    response_format={
        "type": "json_schema",
        "json_schema": {"schema": MergeSchema.model_json_schema(),
        "strict": True},
    },
)
```

Common pitfall — Assuming a provider capability transfers across every model it hosts

A structured-output mode, a tool-calling feature, a context-window size — none of these are provider-wide guarantees; they are per-model capabilities a provider happens to host several of. Code written and tested against `llama-3.3-70b-versatile`'s `json\_schema` support will fail, not degrade, the moment a config change or a cost-driven model swap points the same call at `llama-3.1-8b-instant`. Verify structured-output support per model, the same way Chapter 13B.14's Hugging Face router investigation verified model availability per router rather than assuming Hub presence implied it.

20B.11 Test harness

`test\_output\_fixes.py` runs 302 malformed-input cases across the eight schemas most exercised in production, each case tagged with an identifier that names its failure family at a glance — `PC04-mj` (a Python-class-syntax case against the merge-judge schema), `C18-dc` (an outright refusal message against the DC-scan schema) — so a failing case points straight at both the input shape and the schema it broke against.

Running the suite with per-case timing turns the pipeline's own tiered design into a visible, measured fact rather than an assumption, as Figure 20B.2 shows directly.

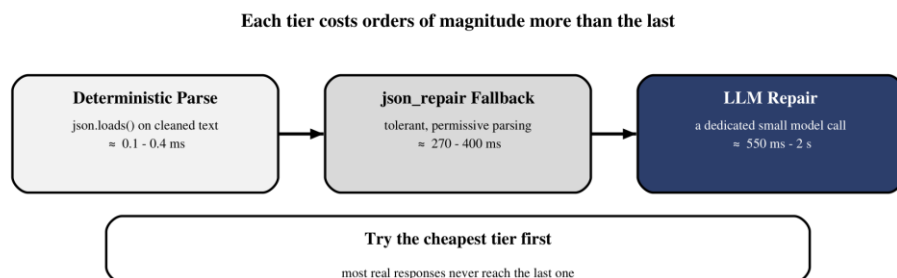


Figure 20B.2 — Elapsed time per case makes the pipeline's own escalation order directly observable.

That measured cost gradient is what proved the layered design earns its complexity: deterministic parsing resolves the large majority of real cases in under half a millisecond, and only a minority ever pay the LLM-repair tier's cost at all. The same suite is also what surfaced Chapter 20.11's fabrication bug in the first place — every Python-class-syntax case returning `{}` instead of escalating, every bare-refusal case producing a populated object instead of signaling absence, found because the harness ran all forty catalogued failure modes against real schemas and logged exactly where each one landed, not because anyone suspected those specific cases in advance.

A test harness built from a failure-mode catalogue rather than from whatever inputs happened to be on hand is the difference between "this passed the tests I thought to write" and "this was checked against every failure this project has ever actually seen." The second claim is the only one worth making about a repair pipeline whose entire job is surviving inputs nobody could fully anticipate.

`fix_llm_output.py` closes the loop this Part opened: Chapter 18 built an agent that could act, Chapter 19 and 19B gave it a graph to act within, Chapter 20 gave it a judge to check what it produced, and this chapter gave every structured call in between a way to survive its own model's imperfect grip on syntax. None of it makes the underlying model more reliable — it makes the system around an unreliable model reliable instead, which is the only kind of reliability a project that does not control model weights ever actually gets to build. Part V turns from correctness to memory: how a system that already retrieves, generates, and judges well begins to learn from its own validated interactions over time.